

****COMPLETE DOCKER NOTES ****

TABLE OF CONTENTS

1. [Introduction & Prerequisites](#)
 2. [1. Getting Started](#)
 3. [2. Developing with Docker](#)
 4. [3. Swarm Introduction](#)
 5. [4. Kubernetes Introduction](#)
 6. [5. CI & Deployment](#)
 7. [Quick Reference Cheatsheet](#)
 8. [Next Steps](#)
-

INTRODUCTION & PREREQUISITES

Why Docker? (Detailed Benefits)

Problem Without Docker	Docker Solution	Real Impact
"Works on my machine"	Identical environments	Zero "it works locally" bugs
Manual setup (hours)	<code>docker run</code> (seconds)	10x faster onboarding
Dependency conflicts	Isolated containers	Run Node 14 + Python 3.9 together
Different OS testing	Any platform	Test Windows app on Linux

Core Philosophy: *Package your app + ALL dependencies into a portable container*

What You ABSOLUTELY Need

1. **Docker Desktop** (Windows/Mac) or **Docker Engine** (Linux)

```
# Verify installation
docker --version
docker run hello-world
```

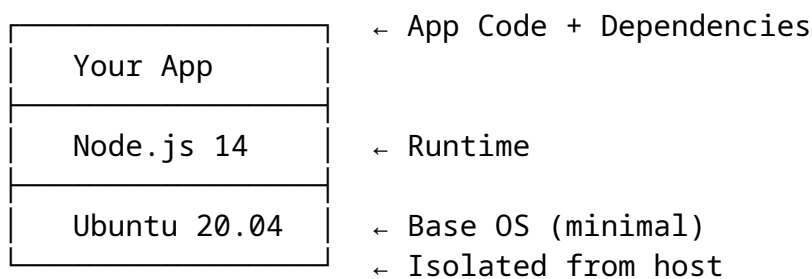
2. **Git:** `git --version`
3. **Basic Terminal:** PowerShell, Bash, or Command Prompt
4. **Any Code Editor:** VS Code recommended

Pro Tip: Always run `docker system prune -a` weekly to clean up space

1. GETTING STARTED

1.1 Docker Concepts & Use Cases for Developers (DETAILED)

The Container Stack (Layer by Layer):



Component	Definition	Developer Workflow	Example
Image	Read-only template	<code>docker build</code>	<code>node:14-alpine</code>
Container	Running instance	<code>docker run</code>	Live Node.js server
Dockerfile	Build instructions	Version control your env	Multi-line recipe
Registry	Image storage	<code>docker push/pull</code>	Docker Hub, private repo
Volume	Persistent data	<code>docker volume create</code>	Database files
Network	Container communication	<code>docker network create</code>	API ↔ Database

5 Real Developer Use Cases:

1. **Local Dev:** `docker run -p 3000:3000 myapp` (instant setup)
2. **Team Sync:** Share exact Dockerfile (no "my npm version is...")

3. **Multi-Version Testing:** Run MySQL 5.7 + 8.0 simultaneously
4. **Microservices:** 10 containers talking via Docker Network
5. **CI/CD:** Build once, deploy everywhere

1.2 Base Application Setup (COMPLETE CODE)

Step 1: Create Project Structure

```
mkdir docker-tutorial && cd docker-tutorial
npm init -y
npm install express
mkdir src
```

Step 2: Complete src/server.js

```
const express = require('express');
const app = express();
const PORT = process.env.PORT || 3000;

app.get('/', (req, res) => {
  res.json({
    message: 'Hello Docker!',
    timestamp: new Date().toISOString(),
    env: process.env.NODE_ENV || 'development'
  });
});

app.get('/health', (req, res) => res.status(200).send('OK'));

app.listen(PORT, () => {
  console.log(`👉 Server running on port ${PORT}`);
  console.log(`👉 Environment: ${process.env.NODE_ENV}`);
});
```

Step 3: Complete package.json

```
{
  "name": "docker-tutorial",
  "version": "1.0.0",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "NODE_ENV=development node src/server.js",
    "test": "echo 'Add tests later'"
  }
}
```

```
    },  
    "dependencies": {  
      "express": "^4.18.2"  
    }  
  }  
}
```

Test Without Docker:

```
npm start  
# Visit http://localhost:3000 ☑
```

1.3 Creating Your First Docker Image (STEP-BY-STEP)

Complete Project Structure:

```
docker-tutorial/  
├── Dockerfile  
├── package.json  
└── src/  
    └── server.js
```

Step 1: Write Dockerfile (EXPLAINED LINE-BY-LINE)

```
# 1. BASE IMAGE: Official Node.js (smallest version)  
FROM node:14-alpine  
  
# 2. WORKING DIRECTORY: Inside container  
WORKDIR /app  
  
# 3. COPY PACKAGE FILES FIRST: Leverage Docker cache  
COPY package*.json ./  
  
# 4. INSTALL DEPENDENCIES: Cached if package.json unchanged  
RUN npm ci --only=production  
  
# 5. COPY SOURCE CODE: Only rebuilds if code changes  
COPY src/ ./src/  
  
# 6. EXPOSE PORT: Documentation only (not publishing)  
EXPOSE 3000  
  
# 7. DEFAULT COMMAND: What runs when container starts  
CMD ["npm", "start"]
```

Step 2: Build Image (Detailed Output)

```
docker build -t my-docker-app .
```

Expected Output:

```
Step 1/7 : FROM node:14-alpine
Step 2/7 : WORKDIR /app
Step 3/7 : COPY package*.json ./
Step 4/7 : RUN npm ci --only=production
Step 5/7 : COPY src/ ./src/
Step 6/7 : EXPOSE 3000
Step 7/7 : CMD ["npm", "start"]
Successfully built abc123
```

Step 3: Run Container (Port Mapping Explained)

```
docker run -p 3000:3000 -e NODE_ENV=production my-docker-app
```

Flags Explained:

- `-p 3000:3000`: Host:Container port
- `-e NODE_ENV=production`: Environment variable
- **Result:** `http://localhost:3000` shows JSON response ☒

Troubleshooting:

Error	Fix
port already in use	<code>docker stop \$(docker ps -q)</code>
no such file	Check COPY paths in Dockerfile
permission denied	<code>sudo chown -R \$USER .</code>

1.4 Base Commands Exploration (COMPLETE GUIDE)

Command	Full Syntax	What It Does	Practical Example	Output Example
<code>docker build</code>	<code>docker build -t name .</code>	Creates image	<code>docker build -t api .</code>	Image ID: abc123
<code>docker run</code>	<code>docker run [flags] image</code>	Starts container	<code>docker run -d -p 80:3000 api</code>	Container running
<code>docker ps</code>	<code>docker ps</code>	List	<code>docker ps</code>	Shows

Command	Full Syntax	What It Does	Practical Example	Output Example
	-a	containers		running IDs
docker stop	docker stop ID	Graceful shutdown	docker stop abc123	Container stopped
docker rm	docker rm ID	Remove stopped	docker rm abc123	Container deleted
docker images	docker images	List images	docker images	Shows all images
docker rmi	docker rmi name	Remove image	docker rmi api	Image deleted
docker logs	docker logs ID	View output	docker logs abc123	Server logs
docker exec	docker exec -it ID bash	Enter container	docker exec -it abc123 sh	Inside container

Power Combo (Run in background + auto-restart):

```
docker run -d --restart always -p 3000:3000 my-docker-app
```

Clean Everything:

```
docker system prune -a --volumes
```

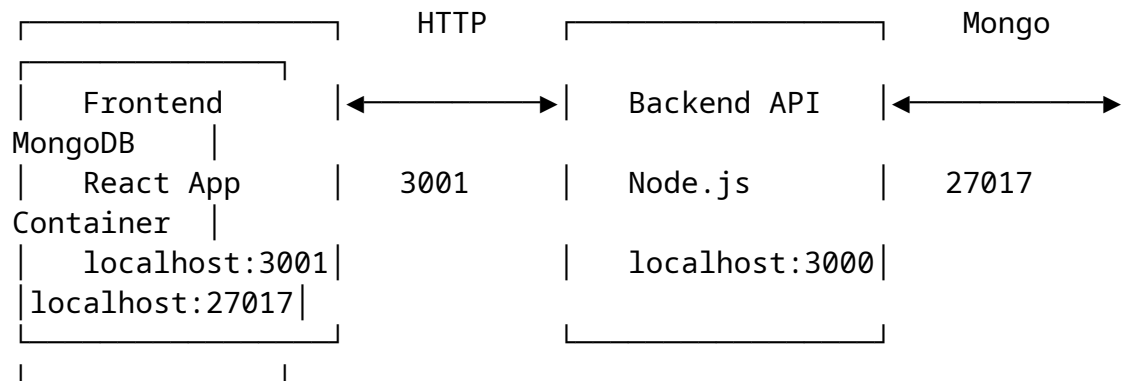
CHAPTER QUIZ ANSWERS:

1. **What does EXPOSE do?** Documents port (doesn't publish). Use -p to publish.
 2. **Image vs Container?** Image = blueprint (static). Container = running instance (dynamic).
-

2. DEVELOPING WITH DOCKER

2.1 Overview: Back-end/Front-end Architecture (DETAILED DIAGRAM)

Complete Architecture:



Communication Flow:

1. User → Frontend (React)
2. React → API calls to Backend
3. Backend → MongoDB queries
4. Response → User

2.2 Practical Backend with Compose (COMPLETE IMPLEMENTATION)

Project Structure:

```

fullstack-app/
├── docker-compose.yml
├── backend/
│   ├── Dockerfile
│   ├── package.json
│   └── src/
│       └── server.js
└── frontend/
    ├── Dockerfile
    └── ... (later)
  
```

Step 1: Backend package.json

```

{
  "scripts": {
    "start": "node src/server.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "mongoose": "^7.0.0"
  }
}
  
```

```
}  
}
```

Step 2: Backend src/server.js

```
const express = require('express');  
const mongoose = require('mongoose');  
const app = express();  
  
mongoose.connect(process.env.MONGO_URL ||  
'mongodb://localhost:27017/myapp')  
  .then(() => console.log('✅ MongoDB Connected'))  
  .catch(err => console.error('❌ MongoDB Error:', err));  
  
const userSchema = new mongoose.Schema({ name: String });  
const User = mongoose.model('User', userSchema);  
  
app.use(express.json());  
  
app.get('/api/users', async (req, res) => {  
  const users = await User.find();  
  res.json(users);  
});  
  
app.post('/api/users', async (req, res) => {  
  const user = new User({ name: req.body.name });  
  await user.save();  
  res.json(user);  
});  
  
const PORT = process.env.PORT || 3000;  
app.listen(PORT, () => console.log(`🚀 Backend on port ${PORT}`));
```

Step 3: Backend Dockerfile

```
FROM node:14-alpine  
WORKDIR /app  
COPY package*.json ./  
RUN npm ci --only=production  
COPY src/ ./src/  
EXPOSE 3000  
CMD ["npm", "start"]
```

Step 4: Complete docker-compose.yml


```

version: '3.8'
services:
  api:
    build: ./backend
    ports:
      - "3000:3000"
    environment:
      - MONGO_URL=mongodb://db:27017/myapp
    depends_on:
      - db
    networks:
      - app-network

  db:
    image: mongo:5.0
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db
    networks:
      - app-network

volumes:
  mongo-data:

networks:
  app-network:
    driver: bridge

```

Step 5: Run & Test

```

docker-compose up --build
# Test: curl http://localhost:3000/api/users

```

Detailed Output Explanation:

```

api_1      | [x] MongoDB Connected
api_1      | [ ] Backend on port 3000
db_1       | MongoDB starting...

```

2.3 Finalize & Test Backend Setup (ADVANCED COMMANDS)

Command	Purpose	Example	What You See
logs	View service	docker-	API startup

Command	Purpose	Example	What You See
	output	<code>compose logs api</code>	logs
<code>exec</code>	Shell access	<code>docker-compose exec api sh</code>	Inside container
<code>scale</code>	Multiple instances	<code>docker-compose up --scale api=3</code>	3 API containers
<code>down</code>	Stop everything	<code>docker-compose down -v</code>	Clean shutdown

Inside Container Debugging:

```
docker-compose exec api sh
# Now inside container:
ls           # See files
npm list     # Check packages
ps aux       # Running processes
mongo       # Connect to DB
```

Load Testing:

```
# Terminal 1: docker-compose up
# Terminal 2:
for i in {1..10}; do curl http://localhost:3000/api/users & done
```

2.4 Practical Frontend Setup (COMPLETE REACT APP)

Frontend Structure:

```
frontend/
├── Dockerfile
├── package.json
├── src/
│   ├── App.js
│   └── index.js
```

Step 1: Frontend package.json

```
{
  "scripts": {
    "start": "react-scripts start"
```

```

    },
    "dependencies": {
      "react": "^18.2.0",
      "react-dom": "^18.2.0",
      "axios": "^1.3.0",
      "react-scripts": "5.0.1"
    },
    "proxy": "http://localhost:3000"
  }
}

```

Step 2: src/App.js

```

import React, { useState, useEffect } from 'react';
import axios from 'axios';

```

```

function App() {
  const [users, setUsers] = useState([]);
  const [name, setName] = useState('');

  useEffect(() => {
    fetchUsers();
  }, []);

  const fetchUsers = async () => {
    const res = await axios.get('/api/users');
    setUsers(res.data);
  };

  const addUser = async () => {
    await axios.post('/api/users', { name });
    setName('');
    fetchUsers();
  };

  return (
    <div style={{ padding: '20px' }}>
      <h1>Users App</h1>
      <input
        value={name}
        onChange={e => setName(e.target.value)}
        placeholder="Enter name"
      />
      <button onClick={addUser}>Add User</button>
    </div>
  );
}

```

```

        <ul>
          {users.map(user => (
            <li key={user._id}>{user.name}</li>
          ))}
        </ul>
      </div>
    );
  }
}

```

export default App;

Step 3: Frontend Dockerfile

```

FROM node:14-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
EXPOSE 3000
CMD ["npm", "start"]

```

Step 4: Update docker-compose.yml

```

frontend:
  build: ./frontend
  ports:
    - "3001:3000"
  depends_on:
    - api
  environment:
    - REACT_APP_API_URL=http://api:3000
  networks:
    - app-network

```

2.5 Finalize & Test Frontend Setup

Complete Testing Flow:

1. docker-compose up --build
2. Visit <http://localhost:3001>
3. Add user → See in list ☒
4. Refresh → Data persists (MongoDB volume) ☒

Environment Variables Deep Dive:

```
frontend:
  environment:
    - REACT_APP_API_URL=http://api:3000 # Container name as hostname
    - NODE_ENV=production
```

Proxy Alternative (in package.json):

```
"proxy": "http://api:3000"
```

Common Issues & Fixes:

Problem	Symptoms	Solution
CORS errors	Frontend can't reach API	Add proxy or use network
Port conflict	3001 in use	Change to 3002:3000
Build fails	Missing deps	docker-compose up --build

2.6 Full Stack Setup: Put It All Together (PRODUCTION READY)

FINAL docker-compose.yml (Copy-Paste Ready):

```
version: '3.8'
services:
  frontend:
    build: ./frontend
    ports: ["3001:3000"]
    depends_on: [api]
    environment:
      - REACT_APP_API_URL=http://api:3000
    networks: [app-network]

  api:
    build: ./backend
    ports: ["3000:3000"]
    depends_on: [db]
    environment:
      - MONGO_URL=mongodb://db:27017/myapp
      - NODE_ENV=production
    networks: [app-network]

  db:
    image: mongo:5.0
```

```
ports: ["27017:27017"]
volumes: [mongo-data:/data/db]
networks: [app-network]
```

```
volumes:
  mongo-data:
```

```
networks:
  app-network:
    driver: bridge
```

Production Commands:

```
# Development
docker-compose up --build

# Production
docker-compose up -d --build
docker-compose logs -f

# Stop
docker-compose down
# Keep data: docker-compose down --remove-orphans
```

Health Check:

```
curl http://localhost:3001          # Frontend
curl http://localhost:3000/api/users # Backend
```

2.7 Other Language Tips & Tricks (COMPLETE EXAMPLES)

Language	Dockerfile	docker- compose.y ml	Run Command	Pro Tips
Python	``dockerfile			
Java	dockerfile FROM openjdk:11 COPY target/app .jar app.jar CMD	yaml app: build: . ports: ["8080:808 0"]	mvn package first	Multi-stage for small image

Language	Dockerfile	docker-compose.yml	Run Command	Pro Tips
PHP	<pre>["java", "-jar", "app.jar"] dockerfile FROM php:7.4- apache COPY . /var/www/h tml</pre>	<pre>yaml app: build: . ports: ["80:80"]</pre>	<pre>docker- compose up</pre>	Apache auto-starts
Go	<pre>dockerfile FROM golang:1.1 9 WORKDIR /app COPY . . RUN go build CMD ["./app"]</pre>	<pre>yaml app: build: . ports: ["8080:808 0"]</pre>	<pre>go mod tidy</pre>	Single binary = tiny image

Multi-Language Monorepo:

```
services:
  python-api:
    build: ./python-api
  node-frontend:
    build: ./node-frontend
  java-worker:
    build: ./java-worker
```

CHAPTER QUIZ ANSWERS:

1. **depends_on:** Starts services in order (db before api)
 2. **environment:** Passes variables to containers
 3. **Background:** `docker-compose up -d`
 4. **Multi-stage:** Copy only runtime artifacts (50% smaller images)
-

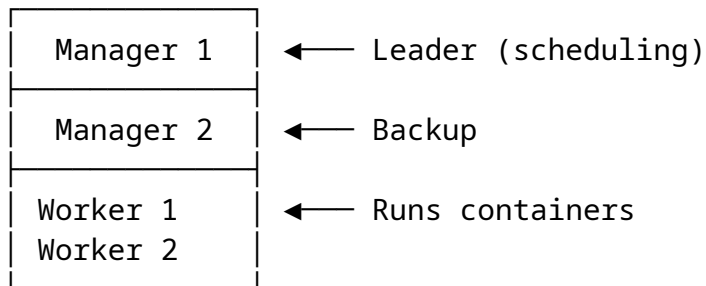
3. SWARM INTRODUCTION

3.1 Overview of Swarm (ARCHITECTURE DEEP DIVE)

Swarm vs Single Docker:

Single Docker	Swarm
1 machine	Multiple machines
Manual scaling	Auto scaling
No load balancing	Built-in LB
Restart on crash	Self-healing

Swarm Architecture:



3.2 Create a Swarm (HANDS-ON SETUP)

Step 1: Initialize Manager

```
# On Machine 1 (Manager)
docker swarm init --advertise-addr 192.168.1.100
```

Output:

```
Swarm initialized: current node is now a manager.
Join token: SWMTKN-1-...
```

Step 2: Join Workers

```
# On Machine 2,3,4 (Workers)
docker swarm join --token SWMTKN-1-... 192.168.1.100:2377
```

Step 3: Verify

```
# On Manager
docker node ls
```


Output:

ID	HOSTNAME	STATUS	ROLE
abc	manager1	Ready	Manager
def	worker1	Ready	Worker

3.3 Adding Nodes (ADVANCED)

Drain Node (maintenance):

```
docker node update --availability drain worker1
```

Promote Worker to Manager:

```
docker node promote worker1
```

Remove Node:

```
docker node rm worker1
```

3.4 Deploy & Inspect Service (COMPLETE EXAMPLE)

Step 1: Create Service

```
docker service create \
  --name webapp \
  --replicas 3 \
  --publish 80:3000 \
  my-docker-app
```

Step 2: Inspect Everything

docker service ls	# All services
docker service ps webapp	# Container locations
docker service logs webapp	# All logs
docker service inspect webapp	# Full config

Step 3: Scale Live

```
docker service scale webapp=5
# Instant! 3→5 containers across nodes
```

Step 4: Update Service

```
docker service update --image my-docker-app:v2 webapp
# Rolling update, zero downtime
```

Remove:

```
docker service rm webapp
docker swarm leave --force # Leave swarm
```

Swarm docker-compose.yml (Add top-level):

```
version: '3.8'
services:
  webapp:
    deploy:
      replicas: 3
      restart_policy:
        condition: on-failure
    image: my-docker-app
    ports: ["80:3000"]
```

Run: `docker stack deploy -c docker-compose.yml myapp`

CHAPTER QUIZ ANSWERS:

1. **Swarm vs Single:** Multi-host orchestration
 2. **--replicas:** Number of container instances
 3. **Remove:** `docker service rm webapp`
-

4. KUBERNETES INTRODUCTION

4.1 Overview (K8s vs Docker vs Swarm)

Feature	Docker	Swarm	Kubernetes
Single Machine	☑	☑	☑
Multi Machine	✗	☑	☑
Auto Scaling	✗	Basic	Advanced
Self Healing	Basic	Good	Excellent
Rolling Updates	✗	Basic	Perfect
Learning Curve	Easy	Medium	Hard

K8s Building Blocks:

- **Pod:** 1+ containers (atomic unit)
- **Deployment:** Manages pods + replicas
- **Service:** Load balancer for pods

- **Namespace:** Virtual clusters

4.2 Installing Kubernetes (Minikube Setup)

Complete Installation:

```
# 1. Install Minikube
curl -LO
https://storage.googleapis.com/minikube/releases/latest/minikube-
linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# 2. Install kubectl
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install kubectl /usr/local/bin/

# 3. Start cluster
minikube start --driver=docker
```

Verify:

```
kubectl cluster-info
kubectl get nodes
# Output: minikube    Ready
```

4.3 Creating Your First Cluster (ADVANCED CONFIG)

Dashboard:

```
minikube dashboard # Opens browser UI
```

Config File (~/.kube/config):

```
kubectl config view
```

4.4 Deploy Your First App (COMPLETE YAML)

Step 1: Create deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
spec:
  replicas: 3
```

```

selector:
  matchLabels:
    app: webapp
template:
  metadata:
    labels:
      app: webapp
  spec:
    containers:
      - name: webapp
        image: my-docker-app:latest
        ports:
          - containerPort: 3000
        env:
          - name: NODE_ENV
            value: "production"
---
apiVersion: v1
kind: Service
metadata:
  name: webapp-service
spec:
  selector:
    app: webapp
  ports:
    - port: 80
      targetPort: 3000
  type: LoadBalancer

```

Step 2: Deploy

```
kubectl apply -f deployment.yaml
```

Step 3: Inspect

```

kubectl get deployments
kubectl get pods
kubectl get services
kubectl describe pod <pod-name>

```

Step 4: Access App

```

minikube service webapp-service
# Opens http://127.0.0.1:xxxxx

```

Step 5: Scale & Update

```
kubectl scale deployment webapp --replicas=5  
kubectl set image deployment/webapp webapp=my-docker-app:v2
```

Debug Commands:

```
kubectl logs <pod-name>  
kubectl exec -it <pod-name> -- sh  
kubectl port-forward <pod-name> 3000:3000
```

Cleanup:

```
kubectl delete -f deployment.yaml  
minikube stop
```

CHAPTER QUIZ ANSWERS:

1. **apply vs create:** Apply updates existing
 2. **Pod:** Smallest deployable unit
 3. **replicas:** Desired pod count
 4. **Logs:** `kubectl logs <pod-name>`
-

5. CI & DEPLOYMENT WITH DOCKER

5.1 CI Use Case for Devs (COMPLETE PIPELINE)

Full Lifecycle:

1. Code Push (GitHub)
↓
2. Trigger CI (Travis)
↓
3. Build Docker Image
↓
4. Run Tests in Container
↓
5. Push Image (Docker Hub)
↓
6. Deploy to Production

5.2 Travis CI File (COMPLETE .travis.yml)

Create .travis.yml:

```
language: node_js
node_js:
  - "14"

services:
  - docker

env:
  - DOCKER_IMAGE=myapp

before_install:
  - echo "$DOCKER_PASS" | docker login -u "$DOCKER_USER" --password-stdin

script:
  # Build
  - docker build -t $DOCKER_IMAGE .
  # Test
  - docker run $DOCKER_IMAGE npm test
  # Tag
  - docker tag $DOCKER_IMAGE $DOCKER_USER/$DOCKER_IMAGE:$TRAVIS_BUILD_NUMBER
  - docker tag $DOCKER_IMAGE $DOCKER_USER/$DOCKER_IMAGE:latest

after_success:
  # Push images
  - docker push $DOCKER_USER/$DOCKER_IMAGE:$TRAVIS_BUILD_NUMBER
  - docker push $DOCKER_USER/$DOCKER_IMAGE:latest

branches:
  only:
    - main
```

Add Tests (package.json):

```
"scripts": {
  "test": "node src/test.js"
}
```

src/test.js:

```
const request = require('supertest');
const app = require('./server');
```

```
describe('API Tests', () => {
  it('should return Hello Docker', (done) => {
    request(app)
      .get('/')
      .expect(200)
      .expect(/Hello Docker/)
      .end(done);
  });
});
```

5.3 Integration into GitHub (STEP-BY-STEP)

1. Push to GitHub:

```
git add .
git commit -m "Add Docker CI"
git push origin main
```

2. Enable Travis:

- o Go to travis-ci.com
- o Sign in with GitHub
- o Enable your repository

3. Add Secrets (Repository Settings → Secrets):

- o DOCKER_USER: yourusername
- o DOCKER_PASS: Docker Hub password
- o DOCKER_TOKEN: Personal Access Token

4. Watch Build:

- o travis-ci.com/yourusername/yourrepo
- o Green = Success → Image on Docker Hub!

5.4 Application Lifecycle with CI & Docker

Production Deploy Script (deploy.sh):

```
#!/bin/bash
IMAGE="youruser/myapp:latest"

echo "🔄 Pulling latest image..."
docker pull $IMAGE
```

```
echo "❑ Stopping old container..."
docker stop myapp-prod || true
docker rm myapp-prod || true
```

```
echo "❑ Starting new container..."
docker run -d \
  --name myapp-prod \
  -p 80:3000 \
  --restart always \
  $IMAGE
```

```
echo "✅ Deployment complete!"
docker logs myapp-prod
```

Run: `chmod +x deploy.sh && ./deploy.sh`

Blue-Green Deploy (Zero Downtime):

```
docker run -d --name myapp-blue -p 80:3000 $IMAGE
# Test blue
docker run -d --name myapp-green $IMAGE
# Switch traffic: update load balancer
```

CHAPTER QUIZ ANSWERS:

1. **.travis.yml**: Defines CI pipeline
 2. **after_success**: Runs when tests pass
 3. **Push to registry**: Enables production deployment
-

❑ QUICK REFERENCE CHEATSHEET

Task	Single Docker	Compose	Swarm	Kubernete s
Start	docker run	up	service create	kubectl apply
Scale	Manual	--scale	scale	scale deployment
Logs	logs ID	logs service	logs service	logs pod
Stop	stop ID	down	rm service	delete deployment

Task	Single Docker	Compose	Swarm	Kubernete s
Build	build	up --build	stack deploy	apply -f yaml

Debug Flow (Always Try This Order):

1. `docker-compose logs` (or `kubectl logs`)
2. `docker-compose exec` (or `kubectl exec`)
3. `docker system prune -a`
4. Check ports: `netstat -tulpn`

Aliases (Add to `~/.bashrc`):

```
alias dc="docker-compose"
alias dcu="docker-compose up -d"
alias dcd="docker-compose down"
alias dps="docker ps -a"
```